COMP11124 Interactive Software Design

Testing and GUI

Learning Outcomes

The main objective this week is to have all to the exercises from Sessions 1-4 personally complete and to have log books completed as a group.

The recommended approach to creating a log book is to generate a markdown file for each session. Using extensions within visual studio code markdown files can be provided with a table of contents, converted to pdf format and viewed within the editor.

Students should set up GitHub repositories to back up their portfolio materials and facilitate sharing.

It is essential that you take this opportunity to get all your files up to date and you should check with lab staff that you are meeting an appropriate level of detail in your logbooks.

The exercises below are not part of the module assessment and do not require any logbook entry. These are provided so that students who are up to date at this time can experiment with useful bonus material.

Topics Covered

Testing programs and running in a window.

Getting Started Task: To keep these exercises within your folder structure, create within the Sessions folder a new folder **session_bonus**, all folders and files for this session will be placed here.

Section 1: Testing

Why test a program? At the end of development you can type test data into a program and check that the response matches expectations. If it does the program is tested and working!

Well, yes! But, there is more to consider:

- Testing can be carried out on the final program and also on individual functions within the program.
- Testing can be carried out with a range of inputs to ensure the output is correct over a wide range of conditions.
- Testing can be carried out with invalid inputs to check that the program handles them.
- Testing can be carried out after program changes to ensure that code sections which previously worked are not affected by modification or refactoring.
- Testing can find multiple faults rather than stopping at the first error.
- Repetitive manual input and interpretation of outputs can be automated.
- The structuring of tests can aid the program design process.

Python has a PyPI package named pytest to facilitate testing with documentation at https://docs.pytest.org/en/stable/ .

Another useful reference for pytest is the Betterstack Pytest guide at https://betterstack.com/tag/pytest

To download useful python packages, use Pip, which is a package manager for python and is included by default in versions 3.4 onward. (If this line does not work the likely cause is that python was not correctly added to the system path on installation. A careful uninstall and reinstallation might be the quickest fix for this). Where alternatives are shown for pip and pytest implementation, the left-hand command is tested on a personal PC and the right-hand command in lab J300 (Paisley)

To confirm that pip is installed

```
pip --version                    or    py -m pip –version
```

Current version is pip 26.01

To load pytest enter

```
pip install -U pytest            or    py -m pip install -U pytest
```

Confirm with

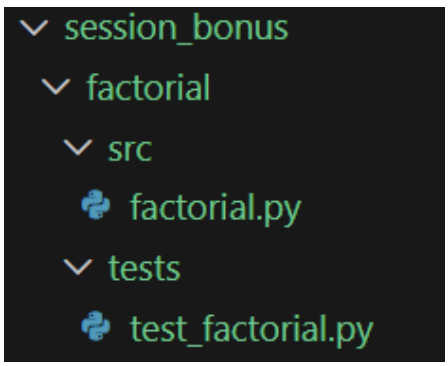
```
pytest --version                 or    py -m pytest
```

Current version is pytest 9.0.2

Exercise 1: Factorial

As a simple example a function to calculate the factorial of a number will be tested.

To keep the test file separate to the source file it is conventional to use a folder structure with a **src** folder and a **tests** folder.



Following this pattern add code to **factorial.py**

```
def factorial(n):
    if n == 1:
        return 1
    return n * factorial(n - 1)

print (factorial(5))
```

This should evaluate 5 * 4 * 3 * 2 * 1 to 120
Running the program shows an output of 120 so it works!

Now lets confirm that by running a test, so in **test_factorial.py** add the code:

```
import pytest
from src.factorial import factorial

def test_factorial_basic():
    assert factorial(5) == 120
```

Always run tests from the root folder not the tests folder so, change directory to **Sessions\session_bonus\factorial** and try

```
python -m pytest
```

This confirms that the factorial of 5 is correctly calculated:

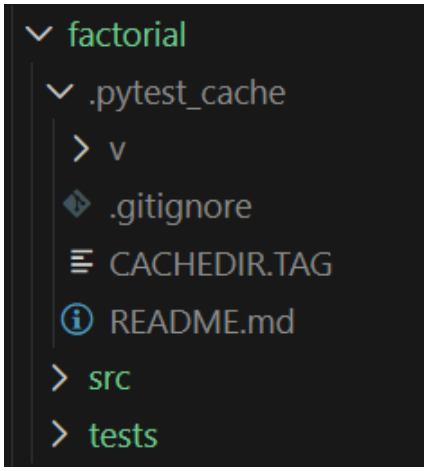
```
========================================== test session starts ========================
platform win32 -- Python 3.14.3, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\... \Sessions\session_bonus\factorial
collected 1 item


tests\test_factorial.py .
[100%]


========================================== 1 passed in 0.02s ==========================
```

This works because the pytest command (run form the root directory) adds to the current working directory any files prefixed with **_test** .

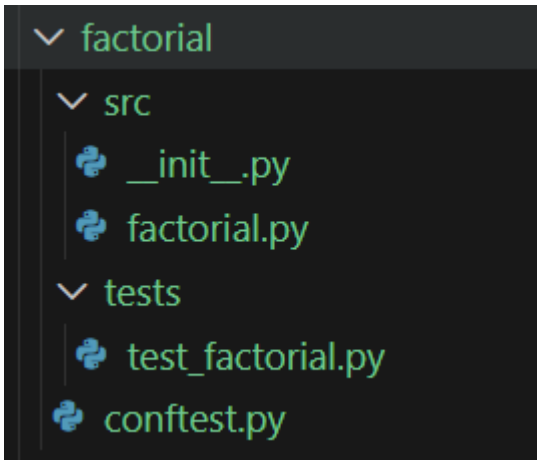
The process of running the test leaves .pytest_cache in the root directory (factorial). This is greyed out because the automatically generatged .gitnore file prevents the contents of this folder from being uploaded to GitHub.



An alternative to using the

```
python -m pytest
```

needs some additional code to make it work. Firstly add a blank file **__init__.py** to the **src** directory, this identifies this as holding a module. Then add **conftest.py** to the root folder (factorial)



The contents of **conftest.py** are

```
import sys
from pathlib import Path

# Add src directory to path
src_path = Path(__file__) / "src"
sys.path.insert(0, str(src_path))
```

What this does is to add the src folder to the python path so that it can be called from within the tests folder.

Now the test can be run from **Sessions\session_bonus\factorial** by entering

```
pytest                                  or      py -m pytest
```

The function works but this is only confirmed for an input of 5. So further tests can be added, perhaps adding a test for input 6.

Think of the cases where the factorial calculation might be vulnerable. Will it work for inputs 0 or 1? Will it work for negative inputs (which do not have a defined factorial)?

Update **test_factorial.py**

```
import pytest
from src.factorial import factorial

def test_factorial_basic():
    assert factorial(5) == 120
    assert factorial(3) == 6

def test_factorial_zero_and_one():
    assert factorial(0) == 1
    assert factorial(1) == 1

def test_factorial_negative_error():
    # This checks that the function correctly raises an error for bad input
    with pytest.raises(ValueError):
        factorial(-1)
```

Before testing, consider why the factorial of zero is one!

Now the test can be run from **Sessions\session_bonus\factorial** by entering

```
Pytest                                    or      py -m pytest
```

This fails spectacularly, in summary:

```
================================ short test summary info ==============================
FAILED tests/test_factorial.py::test_factorial_zero_and_one - RecursionError: maximum
recursion depth exceeded
FAILED tests/test_factorial.py::test_factorial_negative_error - RecursionError: maximum
recursion depth exceeded
========================== 2 failed, 1 passed in 1.80s ==============================
```

Note that we have three tests now and each test can check multiple assertions. Each test gets a fresh instance of the function being tested so failing one test does not prevent later tests from being passed.

To correct these errors the file factorial.py is corrected to:

```
def factorial(n):
    if n < 0:
        raise ValueError("Factorial is not defined for negative numbers.")
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)
```

Running the test then leads to a summary:

```
================================= test session starts =================================
platform win32 -- Python 3.14.3, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\...\Sessions\session_bonus\factorial
collected 3 items


tests\test_factorial.py ...
[100%]


================================== 3 passed in 0.02s ==================================
```

Exercise 2: Temperature

Within the **session_bonus** folder create a new folder named **temperature** and within this create a **src** folder and a **tests** folder.

Add the **conftest.py** file to the **temperature** folder.

```
import sys
from pathlib import Path

# Add src directory to path
src_path = Path(__file__) / "src"
sys.path.insert(0, str(src_path))
```

Copy the temperature conversion programme which was the last file created in Session 2 into the src directory as **src\temperature_converter_function.py**

```
# Temperature Converter
# This program converts temperature from Celsius to Fahrenheit and Kelvin.
scales = ["C", "F", "K"]
temperature_scale = ("Enter the temperature scale you want to convert from: \n 'C'
Celcius \n 'F' Fahrenheit \n 'K'. Kelvin \n")
temperature_scale = input(temperature_scale).strip().upper()
if temperature_scale not in scales:
    print("Invalid scale. Please enter 'C', 'F', or 'K'.")
    exit()
temperature_input = input(f"Enter the temperature in {temperature_scale}: ")
if scales.index(temperature_scale) == 0:  # Celsius
    degree_celcius = float(temperature_input)
    degree_fahrenheit = (degree_celcius * 9/5) + 32
    degree_kelvin = degree_celcius + 273.15
```

```
elif scales.index(temperature_scale) == 1:  # Fahrenheit
    degree_fahrenheit = float(temperature_input)
    degree_celcius = (degree_fahrenheit - 32) * 5/9
    degree_kelvin = (degree_fahrenheit + 459.67) * 5/9
elif scales.index(temperature_scale) == 2:  # Kelvin
    degree_kelvin = float(temperature_input)
    degree_celcius = degree_kelvin - 273.15
    degree_fahrenheit = (degree_kelvin - 273.15) * 9/5 + 32

print("Temperature Conversion Results:")
print(f"{degree_celcius} degree Celsius")
print(f"{degree_fahrenheit} degree Farenheit")
print(f"{degree_kelvin} degree Kelvin")
print("Thank you for using the Temperature Converter!")
```

Run the program to heck that the program works for a known temperature.

Make a blank **__init__.py** in the **src** folder.

To make this program testable it is refactored, making the temperature conversion within a function which is supplied with a letter denoting the temperature scale and a number representing the temperature in degrees to be converted.

src\temperature_converter_function.py refactored

```
# Temperature Converter
# This program converts temperature from Celsius to Fahrenheit and Kelvin.
scales = ["C", "F", "K"]

def convert(temperature_scale:str = "C" , temperature_input:str = "0"):
    if scales.index(temperature_scale) == 0:  # Celsius
        degree_celcius = float(temperature_input)
        degree_fahrenheit = (degree_celcius * 9/5) + 32
        degree_kelvin = degree_celcius + 273.15
    elif scales.index(temperature_scale) == 1:  # Fahrenheit
        degree_fahrenheit = float(temperature_input)
        degree_celcius = (degree_fahrenheit - 32) * 5/9
        degree_kelvin = (degree_fahrenheit + 459.67) * 5/9
    elif scales.index(temperature_scale) == 2:  # Kelvin
        degree_kelvin = float(temperature_input)
        degree_celcius = degree_kelvin - 273.15
        degree_fahrenheit = (degree_kelvin - 273.15) * 9/5 + 32
    return degree_celcius, degree_fahrenheit, degree_kelvin

# The next line prevents the code from being run if called ba a test
if __name__ == "__main__":
    temperature_scale = ("Enter the temperature scale you want to convert from: \n 'C'
Celcius \n 'F' Fahrenheit \n 'K'. Kelvin \n")
    temperature_scale = input(temperature_scale).strip().upper()
    if temperature_scale not in scales:
```

```
        print("Invalid scale. Please enter 'C', 'F', or 'K'.")
        exit()
    temperature_input = input(f"Enter the temperature in {temperature_scale}: ")

    degree_celcius, degree_fahrenheit, degree_kelvin = convert (temperature_scale,
temperature_input)

    print("Temperature Conversion Results:")
    print(f"{degree_celcius} degree Celsius")
    print(f"{degree_fahrenheit} degree Farenheit")
    print(f"{degree_kelvin} degree Kelvin")
    print("Thank you for using the Temperature Converter!")
```

Notice that the function returns three floats which are returned into the values of variables degree_celcius, degree_farenheit and degree_kelvin when the function is called.

The line:

```
if __name__ == "__main__":
```

is included so that if the program is run the code to prompt user input and print results will be executed. However, if the conversion function is imported by another program the user input and output print will not be called. That allows the function to be tested without providing manual input.

Run the conversion program to test that it still works after refactoring as a function.

Following the pattern of testing the factorial function a new file can be added to the **tests** folder as **test_temperature_converter_function.py**

```
import pytest
from src.temperature_converter_function import convert

def test_temperature_converter():
    celsius, fahrenheit, kelvin = convert("C", "0")
    assert celsius == pytest.approx(0, abs=0.01)
    assert fahrenheit == pytest.approx(32.0, abs=0.01)
    assert kelvin == pytest.approx(273.15, abs=0.01)
```

Change directory to **Sessions\session_bonus\temperature** and run the test.

```
Pytest                                  or      py -m pytest
```

```
================================= test session starts =================================
platform win32 -- Python 3.14.3, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\...\session_bonus\temperature
collected 1 item

tests\test_temperature_converter_function.py  .                          [100%]

================================== 1 passed in 0.02s ==================================
```

So, the program works for one input temperature.

In order to test a wider range of inputs to the function a list of parameters can be prepared which are fed in turn into the testing function. **test_temperature_converter_function.py becomes**

```
pytest
from src.temperature_converter_function import convert


@pytest.mark.parametrize(
    "scale, temp, expected_celsius, expected_fahrenheit, expected_kelvin",
    [
        ("C", "0", 0.0, 32.0, 273.15),
        ("F", "32", 0.0, 32.0, 273.15),
        ("K", "273.15", 0.0, 32.0, 273.15),
        ("Z", "273.15", 0.0, 32.0, 273.15),
    ],
)
def test_temperature_converter(scale, temp, expected_celsius, expected_fahrenheit,
expected_kelvin):
    celsius, fahrenheit, kelvin = convert(scale, temp)
    assert celsius == pytest.approx(expected_celsius, abs=0.01)
    assert fahrenheit == pytest.approx(expected_fahrenheit, abs=0.01)
    assert kelvin == pytest.approx(expected_kelvin, abs=0.01)
```

This tests for temperatures submitted for three temperature scales plus one invalid entry.

```
Pytest                                  or      py -m pytest
```

Three tests are passed, but the "Z"scale entry causes a failure.

```
========================================= FAILURES =========================================
_________________ test_temperature_converter[Z-273.15-0.0-32.0-273.15] _________________

scale = 'Z', temp = '273.15', expected_celsius = 0.0, expected_fahrenheit = 32.0,
expected_kelvin = 273.15

    @pytest.mark.parametrize(
        "scale, temp, expected_celsius, expected_fahrenheit, expected_kelvin",
        [
            ("C", "0", 0.0, 32.0, 273.15),
            ("F", "32", 0.0, 32.0, 273.15),
            ("K", "273.15", 0.0, 32.0, 273.15),
            ("Z", "273.15", 0.0, 32.0, 273.15),
        ],
    )
    def test_temperature_converter(scale, temp, expected_celsius, expected_fahrenheit,
expected_kelvin):
>       celsius, fahrenheit, kelvin = convert(scale, temp)
                                      ^^^^^^^^^^^^^^^^^^^^

tests\test_temperature_converter_function.py:20:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

temperature_scale = 'Z', temperature_input = '273.15'

    def convert(temperature_scale:str = "C" , temperature_input:str = "0"):
>       if scales.index(temperature_scale) == 0:  # Celsius
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
E       ValueError: list.index(x): x not in list

src\temperature_converter_function.py:6: ValueError
================================= short test summary info ==================================
FAILED tests/test_temperature_converter_function.py::test_temperature_converter[Z-
273.15-0.0-32.0-273.15] - ValueError: list.index(x): x not in list
============================= 1 failed, 3 passed in 0.11s ==============================
```

Task 1: Modify temperature_converter_function.py so that the entry of an invalid temperature scale will not cause a failure.

Task 2: Determine the minimum physical value for the temperatures which may be entered for each scale. Modify the program so that it will not accept temperatures below the physical minimun value and add a test which will check this.

Exercise 3: To do list

At the close of session 3 a function based to_do_list was created which had the limited functionality to add to a list of tasks, view the list of tasks and to remove a task.

Set up a new folder **session_bonus\to_do_list** with **src** and **tasks** subfolders. Also copy across the files **__init__.py** and **conftest.py** to the appropriate locations.

In the src folder add the **to_do_list.py** file.

```
tasks = []

def add_task(list):
    list.append(input("Add a task: "))


def view_tasks(list):
    count = 0
    for element in list:
        print(f"{count + 1}.{element}")
        count += 1
    print("Total tasks:", len(list), "\n")


def remove_task(list):
    choice = input("Task number to remove: ")
    choice = int(choice)-1
    if not (choice in range(len(list))):
        print("Invalid task number.")
        return
    list.remove(list[choice])


def main():
    while True:

        print("\n")
        print("To-Do List Manager")
        print("1. Add a task")
        print("2. View tasks")
        print("3. Remove a task")
        print("4. Quit")


        choice = input("Enter your choice: ")
```

```
        if choice == "1":
            add_task(tasks)
        elif choice == "2":
            view_tasks(tasks)
        elif choice == "3":
            remove_task(tasks)
        elif choice == "4":
            print("Goodbye!")
            break
        else:
            print("Invalid choice. Please try again. \n")

if __name__ == "__main__":
    main()
```

Here the last two lines cause the main function to be run if the file is run. Test that this file works as expected.

Note that the choices

"1" Add task
"2" View tasks
"3" Remove task
"4" Quit

are strings when they are entered from a manual input.
Note also that removing a task requires a second input value to identify which task to remove. The file has some protection against trying to remove a task index which does not exist.

The functions can't simply be called from the test file, simulated user input must be provided.

A function keyword "**monkeypatch**" is used so that a function in the **src** folder can be temporarily replaced by an anonymous (lambda) function.

The manual input to be tested is added as a list and this is iterated by the next function to mimic the user typing within the builtin function input.

When the tao do list is viewed it is the output stream which is of interest, not values returned by a function. The output stream can be captured as **capture.out** so that the correct assertions can be tested.

The file **test_to_do_list.py** becomes:

```
from src.to_do_list import main, tasks


def test_main_choice_1_add_task(monkeypatch):
    """Test choice 1: Add a task"""
    tasks.clear()
    inputs = iter(["1", "Buy groceries", "4"])
    monkeypatch.setattr('builtins.input', lambda _: next(inputs))
    main()
```

```
    assert "Buy groceries" in tasks


def test_main_choice_2_view_tasks(monkeypatch, capsys):
    """Test choice 2: View tasks"""
    tasks.clear()
    tasks.append("Task 1")
    tasks.append("Task 2")
    inputs = iter(["2", "4"])
    monkeypatch.setattr('builtins.input', lambda _: next(inputs))
    main()
    captured = capsys.readouterr()
    assert "Task 1" in captured.out
    assert "Task 2" in captured.out
    assert "Total tasks: 2" in captured.out

def test_main_choice_3_remove_tasks(monkeypatch, capsys):
    """Test choice 3: Remove tasks"""
    tasks.clear()
    tasks.append("Task 1")
    tasks.append("Task 2")
    inputs = iter(["3", "2", "4"])# remove task 2, view, quit
    monkeypatch.setattr('builtins.input', lambda _: next(inputs))
    main()
    assert "Task 2" not in tasks
```

On running

```
Pytest                                    or      py -m pytest
```

the three tests are all passed.

```
================================= test session starts =================================
platform win32 -- Python 3.14.3, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\derek\Documents\Github\Python-ISD26\Sessions\session_bonus\to_do_list
collected 3 items

tests\test_to_do_list.py ...                                              [100%]

================================== 3 passed in 0.05s ==================================
```

Task1: Add an error into the to_do_list.py file and confirm that the test spots the error.

Task2: Add a test which check that the program will not fail if an invalid choice is made.

Task 3: Refactor the test file so that the choice list is expressed as variables at the top of the program making maintenance easier when further choices are added.

Section 1: GUI programming

There are several libraries which can be used for the creation of Windowed GUI interfaces. One such library is tkinter which is a python interface to Tcl/Tk

Documentation is available at python.org: https://docs.python.org/3/library/tkinter.html#

A useful tutorial which focusses on available widgets is presented by geeksforgeeks: https://www.geeksforgeeks.org/python/python-gui-tkinter/

An extensive tutorial is at python tutorials: https://www.pythontutorial.net/tkinter/

Exercise 1

Try out the following code in **session_bonus\Tkinter\src** as file **to_do_week_3Tk.py**

```
import tkinter as tk
from tkinter import messagebox

tasks = []
root = None
tasks_listbox = None
input_entry = None
input_label = None
current_input_callback = None


def create_main_window():
    global root
    root = tk.Tk()
    root.title('To Do List')
    root.geometry('600x400+50+50')
    root.resizable(True, True)
    root.attributes('-topmost', 1)
    root.columnconfigure(0, weight=1)
    root.columnconfigure(1, weight=4)
    root.rowconfigure(0, weight=1)
    root.rowconfigure(1, weight=0)

    left_frame = create_left_frame(root)
    left_frame.grid(column=0, row=0, sticky="nsew")
    right_frame = create_right_frame(root)
    right_frame.grid(column=1, row=0, sticky="nsew")

    input_frame = create_input_frame(root)
    input_frame.grid(column=0, row=1, columnspan=2, sticky="ew")

    root.mainloop()

def create_left_frame(parent):
```

```
    frame = tk.Frame(parent)
    tk.Label(frame, text="Menu", font=("Arial", 12, "bold")).pack(padx=10, pady=10)

    for text, command in [("Add Task", add_task_gui),
                          ("Remove Task", remove_task_gui),
                          ("Quit", parent.quit)]:
        tk.Button(frame, text=text, command=command).pack(padx=10, pady=5, fill=tk.X)

    return frame


def create_right_frame(parent):
    frame = tk.Frame(parent)
    frame.grid_rowconfigure(2, weight=1)
    frame.grid_columnconfigure(0, weight=1)

    label = tk.Label(frame, text="Tasks", font=("Arial", 12, "bold"))
    label.grid(row=0, column=0, sticky="w", padx=5, pady=5)

    global tasks_listbox
    tasks_listbox = tk.Listbox(frame, font=("Arial", 10))
    tasks_listbox.grid(row=1, column=0, sticky="new", padx=5, pady=5)

    return frame


def create_input_frame(parent):
    global input_label, input_entry
    frame = tk.Frame(parent, bg="lightgray")
    frame.columnconfigure(1, weight=1)

    input_label = tk.Label(frame, text="", font=("Arial", 10), bg="lightgray")
    input_label.grid(row=0, column=0, sticky="w", padx=10, pady=10)

    input_entry = tk.Entry(frame, font=("Arial", 10))
    input_entry.grid(row=0, column=1, sticky="ew", padx=10, pady=10)
    input_entry.bind("<Return>", handle_input_entry)

    return frame


def show_input(prompt):
    """Display an input prompt in the bottom frame"""
    global input_label, input_entry
    input_label.config(text=prompt)
    input_entry.delete(0, tk.END)
    input_entry.focus()


def handle_input_entry(event=None):
    """Handle when user presses Enter in the input field"""
    global current_input_callback
```

```
    if current_input_callback:
        current_input_callback(input_entry.get())
        current_input_callback = None
        hide_input()


def hide_input():
    """Clear the input frame"""
    input_label.config(text="")
    input_entry.delete(0, tk.END)


def update_tasks_display():
    global tasks_listbox
    tasks_listbox.delete(0, tk.END)
    for i, task in enumerate(tasks, 1):
        tasks_listbox.insert(tk.END, f"{i}. {task}")


def add_task_gui():
    global current_input_callback

    def process_task(task):
        if task.strip():
            tasks.append(task)
            update_tasks_display()

    current_input_callback = process_task
    show_input("Enter a new task:")


def remove_task_gui():
    if not tasks:
        messagebox.showinfo("Info", "No tasks to remove")
        return

    def process_removal(task_num_str):
        try:
            task_num = int(task_num_str)
            if 1 <= task_num <= len(tasks):
                tasks.pop(task_num - 1)
                update_tasks_display()
            else:
                messagebox.showerror("Error", f"Invalid task number. Enter 1-
{len(tasks)}")
                remove_task_gui()
        except ValueError:
            messagebox.showerror("Error", "Please enter a valid number")
            remove_task_gui()

    global current_input_callback
    current_input_callback = process_removal
```

```
    show_input(f"Task number to remove (1-{len(tasks)}):")

def main():
    create_main_window()

if __name__ == "__main__":
    main()
```

When create_main_window() is calledfrom main() this example code sets up a frame with with two rows and two colums. A left frame is placed in row 0 column 0. A right frame is placed in row 0 column 1. An input frame is placed on row 1 covering both row 0 and row 1.

The mainloop is started to activate the window,

The left frame holds a label and three buttons. The first button executes a function **add_task_gui** when it is clicked. The second button executes a function **remove_task_gui** when it is clicked. The third button executes parent.quit which closes the window and is equivalent to clicking the x at the top right corner of the window.

The right frame has a label and a listbox which initially is blank, but wich will subsequently show a list of the tasks automatically. (There is no need to have a view tasks button in this case).

The input frame has a label (initially with no text) and a entry widget. When text is entered here it is sent to the function handle_input_entry.

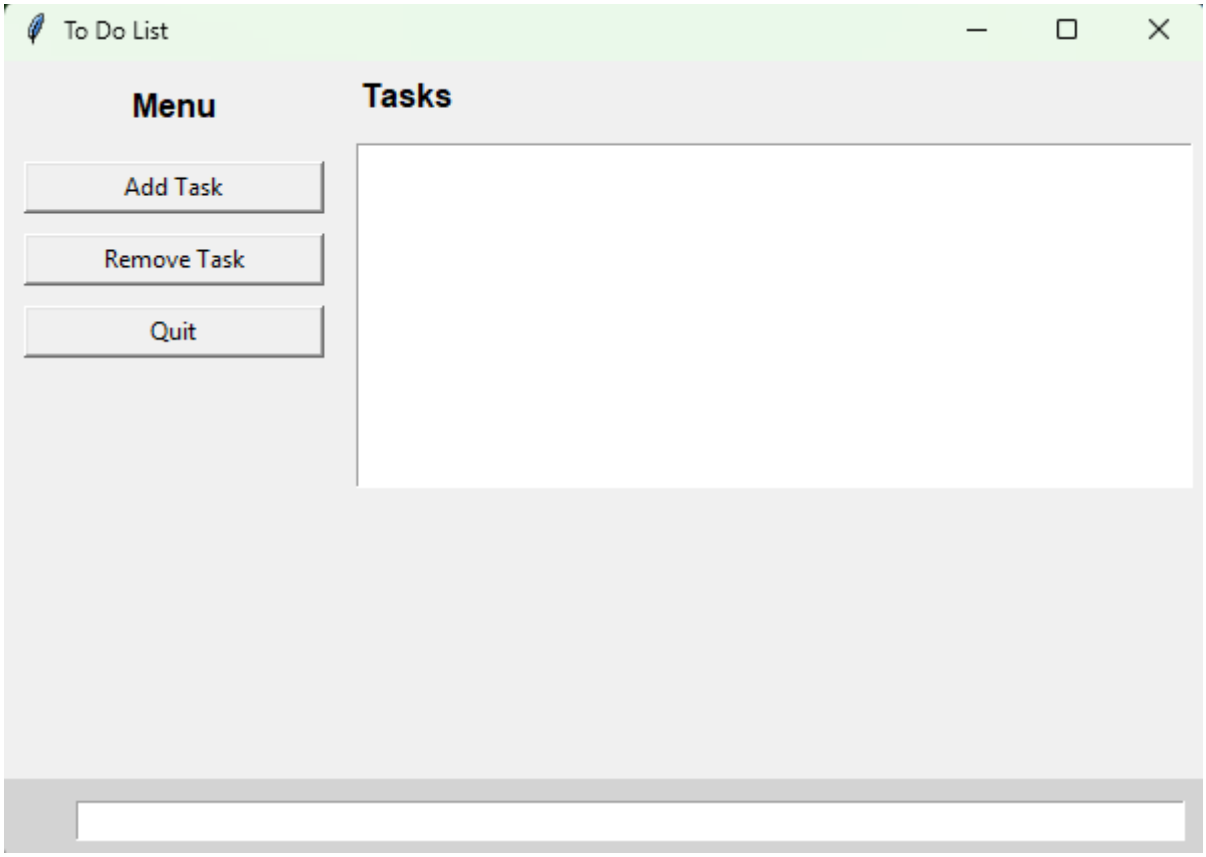
When the **Add Task button** is presssed **add_task_gui()** is called, **process_task** is set to be the **current_input_callback**, that is the function which will be called when the value of the input is changed in response to the prompt to Enter a new task. Show input displays the prompt and places focus onto the input_entry.

When input entry changes **handle_input_entry()** will call the current callback function , **process _task()** which appends the task to the task list and calls **update_tasks_display().**
Finally handle_input_entry calls **hide_input()** to clear the text which has been acted upon.

When the Remove Task button is pressed **remove_task_gui()** is called. This first checks if there are any tasks remaining to be removed. If there are, it pops the indexed task from the task list and updates the talks list and then the **tasks display**().

If the Quit button is pressed the parent window is

When run the programme runs in the window:


To Do List
Menu
Tasks
Add Task
Remove Task
Quit